

Tracing Feedback-Driven Work in Hydro

What causal provenance can establish, and what remains

September 2026

Contents

1	Tracing Feedback-Driven Work in Hydro	2
1.1	Executive summary	2
1.2	1. The systems question	2
1.3	2. The driving hypothesis	3
1.4	3. Ground truth established before provenance	3
1.4.1	3.1 Productive recursion	3
1.4.2	3.2 Timeout and retry	4
1.4.3	3.3 Fixed operational traffic	4
1.5	4. Why aggregate telemetry was insufficient	4
1.6	5. What the provenance implementation does	4
1.6.1	5.1 A concrete example	4
1.6.2	5.2 Generic compiler and runtime support	5
1.6.3	5.3 Opaque user state	5
1.6.4	5.4 Generic post-processing	5
1.7	6. How the experiments were conducted	6
1.8	7. What the experiments established	6
1.8.1	7.1 Retry converts retained obligations into repeated physical work	6
1.8.2	7.2 Productive recursion remains distinguishable	7
1.8.3	7.3 Fixed operational traffic is visible as a separate case	7
1.8.4	7.4 The broader survey finds several distinct replay mechanisms	7
1.9	8. What this says about feasible analysis	8
1.9.1	8.1 Dynamic causal attribution is feasible	8
1.9.2	8.2 Fully automatic semantic classification is not yet feasible from lineage alone	8
1.9.3	8.3 Generic instrumentation does not remove the need for experimental design	8
1.9.4	8.4 Deterministic generic campaign prototype	8
1.9.5	8.5 Buffer-level analysis is promising but incomplete	9
1.9.6	8.5 Collapse prediction remains a separate model	9
1.10	9. The main technical lessons	10
1.11	10. What has and has not been achieved	10
1.12	11. Implementation and reproduction	11
1.13	12. Next steps	11

1 Tracing Feedback-Driven Work in Hydro

1.1 Executive summary

This project asks whether Hydro can identify feedback paths that may sustain overload without mistaking ordinary recursive computation for a hazard. The important distinction is not between cyclic and acyclic programs. It is between a computation that consumes a finite supply of new logical facts and stops, and a mechanism that can repeatedly turn an old logical obligation into new physical work.

We build simulator instrumentation that follows the causal ancestry of each item through supported Hydro IR operators. It records which application inputs and operational events, such as timer firings, contributed to every network send, simulator output, and feedback-cycle crossing. This instrumentation is generic across the IR operators it supports. It does not recognize retries, consensus, gossip, or any other protocol by name.

We then wrote protocol-specific experiments that isolate particular events and inspect the resulting traffic. These experiments show that causal lineage can distinguish a first request from a timer-driven retry, distinguish both from a heartbeat, and preserve productive recursive derivations in transitive closure. They also expose recurring retained-state transmission in gossip, Raft, Multi-Paxos, and a MicroBus client. The experiments are deliberately tailored to each program; we have built a generic measurement mechanism, not an automatic generic test generator.

The strongest conclusion is that causal analysis of feedback-driven work is possible in Hydro's simulator without application-specific annotations inside the programs. The remaining difficulty is semantic. Lineage can show that old inputs contributed to new work, but it cannot always decide whether that work represents a genuinely new fact, a harmless summary, necessary recovery, or waste. Predicting actual metastable collapse additionally requires rates, capacities, and a workload experiment. Provenance supplies a useful layer of that analysis, but not the whole analysis.

1.2 1. The systems question

A metastable failure has two parts. A finite disturbance first pushes a system into a bad operating regime. After the disturbance is removed and the original workload and capacity are restored, internal feedback keeps the system in that regime. Retries are the standard example: overload delays responses, delayed responses cause retries, and retries add enough load to preserve the delay.

Hydro exposes feedback loops in its dataflow graph, but the presence of a loop says little by itself. Consider two programs.

A semi-naive transitive-closure computation repeatedly joins newly discovered paths with known edges:

```

new paths -> join with edges -> candidate paths -> remove known paths -> new paths
      ^                                     |
      +-----+

```

This loop can retain substantial state and produce many intermediate tuples. Nevertheless, each path enters the frontier once. Finite input implies a finite closure, and the computation stops at its fixed point.

A timeout/retry program also contains a loop:

```
request -> attempt -> delayed response -> timeout -> another attempt
```

^ |
+----- more load -----+

The second attempt does not represent a second logical request. An operational event, the timeout, has turned an existing obligation into additional physical work. Whether that mechanism causes collapse depends on

request rates, timeout values, queueing, service capacity, and recovery policy. Its presence is a property of the program execution; collapse is a property of the program in a configured environment.

The project therefore has two separate targets:

1. Mechanism detection. Identify paths that can repeatedly convert retained logical obligations into physical work without requiring new application input.
2. Behavior prediction. Determine whether a particular workload, capacity, and trigger drive such a mechanism into a self-sustaining bad regime.

The work in this report addresses the first target. The existing timeout/retry deployment supplies independent evidence for the second: a healthy baseline becomes overloaded after a finite burst, remains impaired when the baseline arrival rate is restored, drains after organic input is stopped, and is healthy again when the baseline is restarted. This is a weak metastable regime because the finite request population eventually drains when new input is removed. Provenance is not used as the oracle for that result.

1.3 2. The driving hypothesis

The working hypothesis is about how buffers acquire work.

A recursive computation is intrinsically bounded when its feedback buffer admits each logical fact at most once. Transitive closure has this shape: a set difference against the known relation prevents an established path from returning to the frontier. The frontier may be large, but its total contents are bounded by the finite result relation.

A work queue is potentially hazardous when it can admit another physical copy of an obligation it has already admitted. The retry service queue has this shape: each timeout can send another copy of every outstanding request, and the service processes each copy. Nothing at the queue entrance rejects a request identifier it has already processed. If duplicate work arrives faster than capacity is freed, the queue can sustain or amplify overload.

This suggests that an eventual analysis should answer three questions for each buffer or queue:

1. Can the buffer admit work derived from a logical obligation it has already admitted?
2. Can an operational event repeat that admission after application input has stopped?
3. Does the repeated work grow with retained state, and is there a gate that bounds or deduplicates it?

These questions are more useful than assigning one label to an entire program. A single protocol can contain harmless heartbeat traffic, productive replication, bounded recovery, and an unsafe queue at the same time.

1.4 3. Ground truth established before provenance

The experiments were designed around controls whose behavior was established independently of the new instrumentation.

1.4.1 3.1 Productive recursion

The transitive-closure program uses application-aware counters for candidate paths, rejected duplicates, new paths, frontier size, and final output. It demonstrates that a feedback loop can have high, data-dependent amplification and still drain. It is the principal false-positive control for any analysis that treats cycles, retained state, or high work volume as suspicious.

1.4.2 3.2 Timeout and retry

The retry program assigns each logical request a stable identifier and counts physical attempts separately from logical completions. A finite-rate service processes one queued attempt per service pulse. These application-level counters establish when retries create duplicate physical work and how that work consumes service capacity.

A separate wall-clock deployment establishes the behavior of one configuration. With an 80 ms timeout and a 20 ms service interval, a baseline of roughly eight requests per second completes without retrying. A burst of 30 requests pushes the system into a regime where the restored baseline produces more than twice as many attempts as new requests and completions lag arrivals. Stopping input allows the finite population to drain; restarting the original baseline is healthy. This result shows that the retry mechanism can produce history-dependent overload, while also showing that the particular experiment is recoverable under a zero-input anti-trigger.

1.4.3 3.3 Fixed operational traffic

A pure-heartbeat program provides the other essential control. A timer causes one fixed-size message from each member to every member. The work is operational rather than application-driven, but it is bounded by membership and does not recycle retained application state.

These controls establish why traffic counters are insufficient. Transitive closure can do a great deal of useful work; heartbeat can do indefinite timer-driven work; retry can duplicate old obligations. Message counts, byte counts, state access, and the presence of a timer do not identify which case produced a particular message.

1.5 4. Why aggregate telemetry was insufficient

Before adding lineage, we measured stage activations, handoff traffic, retained-state access, network messages, payload bytes, and polling time. Those measurements answered where work happened and how much work occurred. They did not answer which earlier event caused a particular send.

Several benign behaviors looked like retry under aggregate telemetry:

- A service can emit during a window with no new input because it is draining an ordinary queue.
- A full-state gossip message grows in bytes as the state grows, even if the number of messages per timer firing remains fixed.
- Productive work can span several sampling windows and then stop.
- A timer can drive heartbeat, batching, checkpoints, elections, or retries.
- A protocol can carry fresh commands, acknowledgements, heartbeats, and log replay through the same compiled stage.

The missing information was causal connection at item granularity. We needed to know that this network message descended from these application inputs and this timer firing, rather than infer causality from activity in the same time window.

1.6 5. What the provenance implementation does

1.6.1 5.1 A concrete example

Suppose request `r7` enters a client. The simulator assigns that input an identity, represented here as `D7`. The first network attempt carries lineage `{D7}`.

Later, retry timer event `T3` fires while `r7` remains outstanding. The client emits another attempt carrying `{D7, T3}`. The record now establishes two facts directly:

- the new attempt depends on the same application request as the first attempt; and
- an operational event caused the retained request to become physical work again.

If the first attempt was lost, the second may be necessary. If the first attempt was merely slow, the second may be waste. The sender executes the same retry mechanism in both cases. Provenance records that mechanism; recipient state and network delivery determine usefulness in a particular run.

1.6.2 5.2 Generic compiler and runtime support

`SimFlow::with_provenance()` runs an IR transformation before code generation. The transformation wraps each item in `Tagged<T>`, consisting of the original value, a set of source-event identities, and a flag indicating whether the set is an approximation.

Application input ports create data tags. Ports declared with `sim_input_operational()` create operational tags for timers, election triggers, service pulses, and similar events. The pass propagates tags according to operator structure:

- maps and filters preserve the input lineage;
- joins and cross products combine both inputs' lineages;
- folds and reductions accumulate the lineages of their inputs;
- network serialization transports lineage in a side frame while recording the byte count of the original payload;
- network sends, receives, simulator outputs, and cycle crossings append records to an emission log.

`Tagged<T>` compares, hashes, and orders by `T` alone. Operators such as `unique`, `sorting`, and `set difference` therefore behave as they do without instrumentation. A recorded simulator execution has also been replayed successfully with provenance enabled, giving evidence that the instrumentation does not introduce new scheduling choices in the tested path.

The transformation supports a substantial but incomplete subset of Hydro IR. Unsupported node types fail explicitly. The supported and unsupported cases are listed in Section 11.

1.6.3 5.3 Opaque user state

Hydro closures can read or mutate arbitrary Rust state through `by_ref` and `by_mut`. The IR exposes the reference but not which field or element the closure used. When such a closure emits an item, the implementation conservatively attaches all lineage accumulated by the referenced state and marks the result as coarse.

This approximation is important. It preserves causal coverage, but it can merge unrelated items. In the retry service, for example, every response may inherit the lineage of the entire queue. The tool can still show that retained requests and a service pulse contributed to the response, but it may not identify the one request that was popped. Similar loss of precision appears in Raft state machines and `sliced!` blocks.

1.6.4 5.4 Generic post-processing

The emission classifier compares a send with earlier sends from the same sender to the same recipient. Its API labels are:

Label	Plain-language interpretation
Constant	The emission depends on neither application input nor an operational event.
FixedOperational	The emission depends on an operational event but no application data.
Productive	No earlier send on this channel carried the same set, or a superset, of application-input ancestry.
Reactivated	An earlier send already carried that application ancestry, and an operational event contributed to t

Label	Plain-language interpretation
Redundant	An earlier send already carried that application ancestry, without an operational event contributing

The comparison uses whole ancestry sets rather than asking whether each individual source tag has appeared before. This matters for derived facts. If (a, b) and (b, c) were sent separately, the derived path (a, c) depends on a combination no earlier message carried. The classifier treats that combination as new, even though both input edges have appeared individually.

This is a causal rule, not a universal definition of semantic progress. A fold that summarizes two old items can produce the same lineage shape as a join that derives a new fact. Recurrence and program behavior are needed to distinguish those cases.

1.7 6. How the experiments were conducted

The instrumentation and emission classifier are generic over supported IR. The experiments are not generic. Each program has a hand-written driver that chooses inputs, isolates events, waits for the dataflow to become quiet, and asserts protocol-specific outcomes.

Quiescence barriers are central to these experiments. The driver supplies one class of stimulus, waits until the simulator has no more work, and then reads the emission log. This makes the tested phase easy to interpret. It does not automatically discover the right phases for an arbitrary program.

The custom setup for each program includes:

Program	Hand-written experimental choices
Heartbeat	Supply timer events to each member and check per-member fan-out.
Timeout/retry	Admit N requests, fire retry and service timers in selected orders, vary N , and optionally drop selected
Transitive closure	Admit edges in an order compatible with the demo's incremental frontier, fire step events until no o
Gossip	Admit a chosen set at one member, fire selected members' pump events, and compare repeated pum
Raft	Fire a selected election timer, admit commands at the resulting leader, and fire replication heartbeat
Reliable broadcast	Admit messages and a duplicate, then check fan-out, echo traffic, delivery, and drain.
Multi-Paxos	Establish selected leaders, admit commands, then trigger later leadership changes with no new com
Dynamic Raft	Elect a leader, replicate commands, apply a membership change, and inspect a later steady heartbea
MicroBus client	Supply configuration, server status, slot data, timeout events, and stalled-gap keepalive events to the

These scenarios are appropriate for validating causal instrumentation against known mechanisms. They do not constitute an automatic repository audit. A future testing system would need either a scenario specification from the programmer or a way to synthesize meaningful operational events, quiescence points, state sweeps, and progress oracles.

1.8 7. What the experiments established

1.8.1 7.1 Retry converts retained obligations into repeated physical work

For N outstanding requests, the initial sends carry the requests' data lineage and are first sends on their channels. A retry event then produces N additional request messages. Each additional send carries retained request lineage plus the retry event. One service pulse completes one request, after which the next retry event produces $N - 1$ request messages.

The result held for $N = 1, 4, 9$. The number of messages caused by one retry event therefore tracks the outstanding-request set rather than being a fixed timer cost.

A second experiment lets k retry events fire before the service drains. For $N = 4$, the service emits $N(k + 1)$ physical responses: 8, 12, and 16 responses for $k = 1, 2, 3$. Only four response payloads are distinct, and the program reports four logical completions. The extra service work is exactly $N*k$: 4, 8, and 12 responses. This directly demonstrates closed-loop amplification in the tested execution: duplicate input at the service consumes capacity and produces duplicate output.

Two loss scenarios clarify the interpretation:

- When odd-numbered responses are permanently discarded, each later retry event re-sends the two unresolved requests. Each pair consumes two service pulses and no further logical request completes. The work recurs for as many timer events as the driver supplies.
- When the service discards the first arrival of odd-numbered requests, later retries are necessary for completion. They still have the same causal shape: the client retained an obligation and a timer caused it to send again. Provenance identifies the retry mechanism without claiming that every retry was useless.

1.8.2 7.2 Productive recursion remains distinguishable

The transitive-closure experiment emits six reachable pairs. Every output carries either a new input edge or a combination of input edges no earlier output carried. After the frontier is exhausted, further step events emit nothing. Re-admitting the same graph also emits nothing because the known-set gate rejects established paths.

This control shows that lineage does not reduce “new work” to “contains a source tag never seen before.” A derived fact may be new because it combines previously separate inputs. More importantly, the computation stops when no such fact remains.

1.8.3 7.3 Fixed operational traffic is visible as a separate case

In a three-member heartbeat cluster, each timer round produces nine messages, one for every sender-recipient pair. Those messages carry the timer event and no application-data lineage. Their cost is fixed by membership rather than retained application state.

Raft vote requests before any commands have arrived show the same causal shape. An election event produces fixed operational traffic. This separates timer-driven control traffic from timer-driven replay of application state.

1.8.4 7.4 The broader survey finds several distinct replay mechanisms

The remaining programs are observations rather than independent ground-truth classifications. They demonstrate the range of behavior visible to lineage tracking.

Program	Observed behavior
G-Set gossip	Initial updates are sent once. A first pump sends the combined set; later pumps send the combined set.
Reliable and uniform broadcast	For two messages and three members, all 24 network sends are first sends on their sender's side.
Raft	Election traffic before data is operational-only. The first heartbeat after commands carries no data.
Dynamic Raft	Election, command replication, membership change, and steady heartbeat have the same causal shape.
Multi-Paxos	A leadership change with no new commands causes each acceptor to send its retained commands.
MicroBus catchup client	Open and timeout/reopen messages are fixed 44-byte operational traffic. A stalled gap in the data stream causes a replay of the entire data stream.

The survey supports a useful distinction between fixed operational control, replay whose payload scales with retained state, productive catch-up that drains, and duplicate work that consumes downstream capacity. It does not establish that every observed replay is dangerous.

1.9 8. What this says about feasible analysis

1.9.1 8.1 Dynamic causal attribution is feasible

For supported Hydro IR, the simulator can answer a question aggregate telemetry could not answer: which external inputs and operational events contributed to this particular emission? The answer is obtained without naming protocols or adding application annotations to their internal logic. This creates a reusable experimental substrate for feedback research.

The result is stronger than a collection of protocol-specific counters. The timeout/retry counters know what a request identifier and an attempt mean; provenance does not. Nevertheless, provenance independently recovers the fact that a timer caused retained request ancestry to cross the request channel again.

1.9.2 8.2 Fully automatic semantic classification is not yet feasible from lineage alone

Causal ancestry is not the same as logical meaning. A join can combine old inputs into a genuinely new fact. A fold can summarize old inputs without creating a new logical obligation. An opaque Rust state machine can emit one item while exposing only the ancestry of the whole state. Those cases may have identical tag sets.

A useful analyzer therefore needs more than lineage. At minimum, it needs recurrence after application input stops, payload or state-change observations when lineage is coarse, and structural knowledge about gates on the relevant path. Some protocols may also require a programmer-supplied progress definition.

1.9.3 8.3 Generic instrumentation does not remove the need for experimental design

The current drivers deliberately manufacture interpretable phases. They know which timer elects a leader, which input is an application command, when a service queue should drain, and what output constitutes completion. The instrumentation can be reused unchanged, but a new protocol still needs a meaningful driver and oracle.

Automation is possible along two axes. A programmer could declare operational inputs and progress outputs, allowing a generic harness to sweep retained state and repeat operational events. Alternatively, the compiler could synthesize candidate experiments from timer sources, feedback edges, and quiescence.

1.9.4 8.4 Deterministic generic campaign prototype

A first generic campaign now exists in `hydro_lang::sim::feedback_campaign`. It does not read prose, protocol names, or test assertions. For every registered flow it executes the same deterministic algorithm:

1. grow a data input through cumulative scales 1, 8, and 32;
2. wait for the dataflow to quiesce after each growth action;
3. fire one selected operational input repeatedly;
4. stop after two identical physical-emission signatures, or after a common maximum of 64 firings;
5. derive bounded witnesses for operational-only traffic, replayed application ancestry, recurring payloads, input-driven scaling, and operational scaling with retained data.

The program adapter is limited to typed boundary mechanics: it maps integer indices to legal input values and fires a typed operational handle. Existing tests are used as executable specifications for these mechanics.

For example, they establish that the transitive-closure input value is a `Vec<Edge>`, that retry requests require stable identifiers, and that gossip updates target a cluster member. They do not provide phases, expected provenance labels, or conclusions to the runner.

`SimFlow::feedback_boundary_manifest()` independently inventories external inputs, input roles, outputs, locations, debug type descriptions, and cycle sinks from Hydro IR. Typed simulator handles expose their numeric port IDs, and `assert_inputs_covered()` rejects an adapter that silently omits a discovered input. The retry evaluation exercises this with all three of its inputs: requests, retry ticks, and service ticks. The current campaign selects one operational port as its probe while explicitly accounting for the others.

One unchanged campaign was run on four flows:

Flow	Evidence produced by the generic runner
Pure heartbeat	Operational-only physical work; no replay of application ancestry and no data-scale effect.
Timeout/retry	Repeated application ancestry after retry events; physical work from one retry grows with the outstar
G-Set gossip	Repeated application ancestry after pump events; serialized work grows with retained set size.
Transitive closure	Data-admission work grows with graph size, but established output ancestry is not replayed; operatio

This is the first result in which the workload pattern and witness extraction are shared across implementations. It reduces the risk that a hand-authored phase script defines the observed distinction. It does not eliminate adaptation: legal typed values, target members, and one selected operational port are still registered per flow. Internal wall-clock timers are still not controllable, and independent control/trigger instances are not yet part of the campaign.

1.9.5 8.5 Buffer-level analysis is promising but incomplete

A prototype `buffer_table` groups emission records by edge and sender-recipient channel. It reports whether the selected execution sent previously sent ancestry again and whether repeated ancestry grew as the driver increased retained state. On the three constructed scenarios it produces the expected contrast:

Scenario and observed edge	Repeated old ancestry?	Scaling in the chosen run
Growing retry backlog, request edge into service	Yes, after retry events	Grew from the smaller to the larger outstar
Transitive-closure output	No	No repeated admission observed
Repeated gossip pump from one member	Yes, after pump events	Same retained set on successive pumps

This prototype is a report over a chosen experiment, not a complete buffer decision procedure. The test currently supplies gate information rather than deriving it from IR. A “grows with state” result depends on a driver that actually varies retained state. Cycle-sink records are available, but interpreting an accumulator carrying state across ticks requires care: repeated state carry is not automatically repeated external work. These are the next analysis problems, not details already solved by the table.

1.9.6 8.5 Collapse prediction remains a separate model

The presence of retry-shaped replay establishes a possible source of extra work. It does not determine whether that work overwhelms a deployment. Collapse depends on at least:

- the arrival rate of new logical work;
- the distribution of service time and available capacity;
- timeout and retry schedules;

- queue discipline and queue bounds;
- cancellation, acknowledgement, and deduplication behavior;
- trigger magnitude and duration; and
- how the feedback mechanism changes load or capacity after the trigger.

The simulator lineage log can measure work generated by a controlled event. A queueing model, discrete-event model, or paired deployment experiment must connect that work to capacity and persistence. Keeping these layers separate allows one mechanism analysis to be reused across many configurations without pretending that all configurations fail.

1.10 9. The main technical lessons

1. A particular send can be causally attributed. Per-item lineage succeeds where time-window correlation failed.
2. A new result can use only old inputs. The relevant question is whether an earlier message carried that combination, not whether every individual source tag has appeared somewhere before.
3. Repeated sending and recipient usefulness are different questions. A sender retries because it lacks evidence of completion. Whether the first copy was lost or merely delayed changes the retry's usefulness, not the sender-side mechanism.
4. Downstream cost matters. Retry becomes operationally important because duplicate requests consume service pulses and produce duplicate responses. Repeated output without downstream amplification, as in the measured gossip experiment, has a different risk profile.
5. Opaque state is the precision boundary. Once arbitrary Rust code reads a large state object, the IR cannot identify which element funded an output. Payload identity and application gates become necessary fallbacks.
6. The useful unit is smaller than a program. Raft and Paxos contain fixed control traffic, productive replication, and retained-state replay. Analysis should report paths and buffers rather than assign one verdict to the protocol.
7. A dynamic mechanism report and a collapse verdict answer different questions. The first can be largely program-structural and execution-causal. The second is quantitative and configuration-dependent.

1.11 10. What has and has not been achieved

The current implementation supports the following claims:

- Hydro's simulator can propagate causal lineage through the supported IR operators and attach it to physical emissions.
- The tested instrumented executions preserve ordinary value comparison and replay behavior.
- In hand-written, barrier-separated experiments, lineage distinguishes initial requests, timer-driven retries, fixed heartbeat traffic, and productive transitive-closure output.
- The retry experiments measure backlog-proportional replay and downstream duplicate work.
- The same instrumentation exposes retained-state replay and productive catch-up phases in several realistic protocols.
- Per-edge reports are technically possible and are a plausible interface for a future buffer-bounding analysis.

The current implementation does not support the following claims:

- It does not automatically generate a meaningful experiment for an arbitrary Hydro program.
- It does not cover every Hydro IR node or external system.
- It does not recover precise item identity after arbitrary opaque state access.
- It does not infer the semantic progress condition of every protocol.

- It does not yet infer deduplication gates on a path.
- It does not decide which cycle-sink state carries correspond to externally costly work.
- It does not predict rates, capacity exhaustion, recovery, or metastable collapse.
- It does not establish that gossip, Raft, Paxos, or MicroBus is unsafe.

1.12 11. Implementation and reproduction

The principal implementation files are:

- `hydro_lang/src/sim/provenance.rs`: tagged values, network framing, the emission log, emission classification, attribution, and the prototype buffer table;
- `hydro_lang/src/sim/provenance_ir.rs`: the compiler pass that propagates lineage through Hydro IR;
- `hydro_lang/src/sim/feedback_campaign.rs`: boundary-manifest types, the deterministic retained-state campaign, epoch measurements, and bounded witness extraction;
- `hydro_test/src/cluster/provenance_generic_campaign.rs`: one unchanged campaign applied to heartbeat, retry, gossip, and transitive closure;
- `hydro_test/src/cluster/provenance_ground_truth.rs`: nine program-specific experiments for heartbeat, retry variants, transitive closure, gossip, and Raft;
- `hydro_test/src/cluster/provenance_survey.rs`: experiments for reliable broadcast, uniform broadcast, Multi-Paxos, and dynamic Raft;
- `hydro_test/src/cluster/provenance_buffers.rs`: three scenarios exercising the prototype per-edge report;
- `design_docs/reports/microbus_probe/`: the extracted MicroBus client probe and its output; and
- `hydro_test/src/distributed/timeout_retry.rs`: the independent wall-clock retry experiment and application-level oracle.

On the development machine, the tests require the macOS 26.5 SDK:

```
SDKROOT=/Library/Developer/CommandLineTools/SDKs/MacOSX26.5.sdk \
cargo test -p hydro_lang --features sim --lib provenance
```

```
SDKROOT=/Library/Developer/CommandLineTools/SDKs/MacOSX26.5.sdk \
cargo test -p hydro_test --lib provenance
```

The first command runs the provenance unit tests. The second runs the ground-truth, survey, and buffer experiments. Classified logs and buffer tables are written to `hydro_test/target/provenance_dump.txt` when the test harness enables the dump.

The current pass rejects unsupported nodes with a clear provenance: `unsupported ... panic`. Unsupported cases include futures-resolution nodes, blocking stream flattening and scans, keyed watermark reduction, versioned networks, raw-byte channels, and external ports. Wall-clock interval sources also cannot be driven by the simulator; tests expose relevant timers as `sim_input_operational` ports.

1.13 12. Next steps

The next milestone should turn the current experimental substrate into a more automatic buffer analysis without mixing it with collapse prediction.

1. Control internal timers. Rewrite simulator builds of internal interval sources into hidden operational inputs and list them in the boundary manifest. Until then, programs must expose timers through `sim_input_operational` to participate.

2. Generalize deterministic adapters. Register a typed value generator, target-member policy, and explicit waiver or probe policy for every discovered input. Keep the adapter declarative: it may define legal values, but not phases or expected findings.
3. Add independent campaign instances. Run geometric scales and paired control/trigger suffixes in fresh simulator instances rather than accumulating every scale in one state. Preserve both workload inputs and scheduler decisions for replay.
4. Evaluate held-out programs. Apply the unchanged campaign to reliable broadcast, Multi-Paxos recovery, Raft catch-up, and at least one additional retry-like system. Declare expected path-level evidence before inspecting the results.
5. Infer gates from IR. For each candidate emission edge, determine whether every path from an operational source passes through a relevant unique, set difference, acknowledgement check, or other state-advance condition. The analysis must identify the actual feedback path rather than merely notice that such an operator exists somewhere in the program.
6. Improve precision through state. Preserve per-key or per-element lineage through common state containers, and use payload identity where arbitrary closures prevent that precision.
7. Measure downstream gain. Extend the generic witness vocabulary from repeated sends to the physical work those sends causally induce at recipients. This separates fixed publication from capacity-consuming feedback.
8. Model collapse separately. Feed measured work gain into workload and capacity experiments. A mechanism report should identify where extra work can arise; a behavior model should determine whether it is enough to sustain overload.

The immediate opportunity is therefore concrete. Hydro can provide causally precise execution evidence about feedback-driven work. With structural gate analysis and a declared experiment interface, that evidence can plausibly become a practical per-buffer diagnostic. Predicting metastability will still require a quantitative model above it, but the causal layer no longer has to guess why a message was sent from aggregate counters.